

- **Họ tên:** Hồ Tuấn Huy
- **MSSV:** 20225856

Báo cáo kết quả bài tập Type_checking buổi 2

I. Mô tả các hàm compileArgument và compileArguments

1. Hàm compileArgument

- **Mô tả chung:** Hàm **compileArgument(Object* param)** dùng để **biên dịch và kiểm tra một đối số** khi gọi hàm hoặc thủ tục.
- **Tham số: param:** Con trỏ đến đối tượng tham số tương ứng trong phần khai báo hàm/thủ tục.
- **Chức năng:** Hàm thực hiện phân tích đối số dựa trên **kiểu tham số** đã khai báo:
 - Xử lý tham số giá trị (PARAM_VALUE):
 - Chấp nhận mọi loại biểu thức: Số, ký tự, biến, lời gọi hàm, ...
 - Gọi **compileExpression()** để phân tích biểu thức và lấy kiểu dữ liệu của đối số.
 - So sánh kiểu đối số với kiểu tham số bằng **checkTypeEquality()**.
 - Xử lý tham số tham chiếu (PARAM_REFERENCE):
 - Chỉ chấp nhận **lvalue**, bao gồm: Biến, tham số, tên hàm.
 - Gọi **compileLValue()** để phân tích value và lấy kiểu dữ liệu tương ứng.
 - Kiểm tra tính tương thích kiểu bằng **checkTypeEquality()**.

- **Code:**

```
void compileArgument(Object* param) {
    // phân tích một đối số và kiểm tra tính nhất quán kiểu
    // nếu tham số tương ứng là tham chiếu, đối số phải là lvalue
    Type* argType;

    if (param->paramAttrs->kind == PARAM_VALUE) {
        argType = compileExpression();
        checkTypeEquality(argType, param->paramAttrs->type);
    } else {
        argType = compileLValue();
        checkTypeEquality(argType, param->paramAttrs->type);
    }
}
```

2. Hàm compileArguments

- **Mô tả chung:** Hàm **compileArguments(ObjectNode* paramList)** dùng để:
 - Biên dịch **danh sách đối số** khi gọi hàm/thủ tục.
 - Kiểm tra tính nhất quán giữa **đối số** và **tham số khai báo**.
- **Tham số: paramList:** Con trỏ đến danh sách liên kết các tham số trong khai báo hàm/thủ tục.
- **Chức năng:**
 - Trường hợp có đối số (gặp dấu (– SB_LPAR):
 - Parse dấu (.
 - Nếu **paramList == NULL**: Báo lỗi **ERR_PARAMETERS_ARGUMENTS_INCONSISTENCY** (thừa đối số).
 - Với mỗi đối số:
 - Gọi **compileArgument()** để phân tích và kiểm tra.
 - Di chuyển sang tham số tiếp theo.
 - Nếu gặp dấu phẩy: Tiếp tục phân tích đối số tiếp theo.
 - Nếu hết đối số nhưng vẫn còn tham số: Báo lỗi **ERR_PARAMETERS_ARGUMENTS_INCONSISTENCY** (thiếu đối số).
 - Parse dấu).
 - Trường hợp không có đối số:

- Kiểm tra các token thuộc tập **FOLLOW** (toán tử, dấu phẩy, dấu đóng ngoặc, từ khóa điều khiển, ...).
- Nếu **paramList != NULL**: Báo lỗi **ERR_PARAMETERS_ARGUMENTS_INCONSISTENCY**.
- Xử lý lỗi cú pháp: Nếu không rơi vào các trường hợp hợp lệ trên: Báo lỗi **ERR_INVALID_ARGUMENTS**.

- **Code:**

```
void compileArguments(ObjectNode* paramList) {
    // phân tích danh sách đối số, kiểm tra tính nhất quán giữa đối số và tham số
    ObjectNode* paramNode = paramList;

    switch (lookAhead->tokenType) {
    case SB_LPAR:
        eat(SB_LPAR);
        if (paramNode == NULL)
            error(ERR_PARAMETERS_ARGUMENTS_INCONSISTENCY, currentToken->lineNo, currentToken->colNo);
        compileArgument(paramNode->object);
        paramNode = paramNode->next;

        while (lookAhead->tokenType == SB_COMMA) {
            eat(SB_COMMA);
            if (paramNode == NULL)
                error(ERR_PARAMETERS_ARGUMENTS_INCONSISTENCY, currentToken->lineNo, currentToken->colNo);
            compileArgument(paramNode->object);
            paramNode = paramNode->next;
        }

        if (paramNode != NULL)
            error(ERR_PARAMETERS_ARGUMENTS_INCONSISTENCY, currentToken->lineNo, currentToken->colNo);

        eat(SB_RPAR);
        break;
        // kiểm tra tập FOLLOW
    case SB_TIMES:
    case SB_SLASH:
    case SB_PLUS:
    case SB_MINUS:
    case KW_TO:
    case KW_DO:
    case SB_RPAR:
    case SB_COMMA:
    case SB_EQ:
    case SB_NEQ:
    case SB_LE:
    case SB_LT:

    case SB_GE:
    case SB_GT:
    case SB_RSEL:
    case SB_SEMICOLON:
    case KW_END:
    case KW_ELSE:
    case KW_THEN:
        if (paramNode != NULL)
            error(ERR_PARAMETERS_ARGUMENTS_INCONSISTENCY, currentToken->lineNo, currentToken->colNo);
        break;

    default:
        error(ERR_INVALID_ARGUMENTS, lookAhead->lineNo, lookAhead->colNo);
    }
}
```

II. Ứng dụng của các hàm trong kiểm tra tương ứng kiểu giữa khai báo và sử dụng hàm, thủ tục

1. Quy trình kiểm tra

- 2 hàm **compileArguments** và **compileArgument** phối hợp để đảm bảo tính nhất quán giữa khai báo và sử dụng hàm/thủ tục, bao gồm cả **số lượng** và **kiểu dữ liệu** của đối số.
- Kiểm tra số lượng đối số:
 - Nếu gặp dấu (nhưng danh sách tham số rỗng => báo lỗi
ERR_PARAMETERS_ARGUMENTS_INCONSISTENCY (thừa đối số).
 - Nếu parse hết đối số nhưng vẫn còn tham số => báo lỗi
ERR_PARAMETERS_ARGUMENTS_INCONSISTENCY (thiếu đối số).
 - Nếu không có đối số nhưng vẫn còn tham số (kiểm tra FOLLOW set) => báo lỗi
ERR_PARAMETERS_ARGUMENTS_INCONSISTENCY.
- Kiểm tra từng đối số:
 - Mỗi đối số được phân tích bởi **compileArgument()** theo loại tham số (giá trị hoặc tham chiếu).
 - Kiểu của đối số được so sánh với tham số tương ứng bằng **checkTypeEquality()**.
 - Nếu **compareType()** cho kết quả không khớp => báo lỗi **ERR_TYPE_INCONSISTENCY**.

2. Các trường hợp đặc biệt

- Tham số tham chiếu (PARAM_REFERENCE):
 - Chỉ chấp nhận **lvalue hợp lệ** (biến, tham số, tên hàm).
 - Đối số được phân tích bằng **compileLValue()**, hàm này kiểm tra khai báo, phạm vi và truy cập mảng nếu có.
 - Kiểu lvalue được so sánh với kiểu tham số bằng **checkTypeEquality()**.
- Tham số giá trị (PARAM_VALUE):
 - Chấp nhận mọi biểu thức hợp lệ (hằng, biến, hàm, biểu thức toán học).

- Đối số được phân tích bằng **compileExpression()** để xác định kiểu, sau đó so sánh với tham số.

3. Luồng xử lý tổng quát

- Khi gặp lời gọi hàm/thủ tục, **compileArguments()** được gọi để xử lý danh sách đối số.
- Nếu có dấu (, các đối số được phân tích lần lượt bằng **compileArgument()**.
- Nếu không có đối số, FOLLOW set được kiểm tra để đảm bảo không thiếu tham số.
- Sau khi hoàn tất, chương trình xác nhận sự khớp đúng về **số lượng** và **kiểu dữ liệu** giữa đối số và tham số.

III. Kết quả thực hiện với example3.kpl

- File **example3.kpl** là chương trình giải bài toán **Tower of Hanoi**, sử dụng procedure **HANOI** với 3 tham số kiểu **INTEGER** và được biên dịch thành công.
- Procedure **HANOI** được khai báo với các tham số **N, S, Z** (đều kiểu **INTEGER**) và được gọi trong chương trình, ví dụ:
 - **CALL HANOI(N-1, S, 6-S-Z)**
 - **CALL HANOI(N, P, Q)**
- Khi gặp lời gọi procedure, compiler gọi **compileArguments** để kiểm tra số lượng đối số, bảo đảm đúng 3 đối số tương ứng với 3 tham số.
- Sau đó, **compileArgument** được gọi cho từng cặp (*tham số, đối số*) để kiểm tra kiểu dữ liệu.
- Do các đối số **N, P, Q** đều có kiểu **INTEGER**, hàm **compareType** trả về kết quả khớp kiểu cho tất cả các cặp.

- Vì không phát hiện lỗi nào, chương trình được **biên dịch thành công**.

```

● → Bai5_Type_Code_Gen ./semantics tests/example3.kpl
Program EXAMPLE3 at address 0
  Var I : Int at offset 4
  Var N : Int at offset 5
  Var P : Int at offset 6
  Var Q : Int at offset 7
  Var C : Char at offset 8
  Procedure HANOI at address 1
    Param N : Int at offset 4
    Param S : Int at offset 5
    Param Z : Int at offset 6

○ → Bai5_Type_Code_Gen █

```

IV. Thay đổi trong các ví dụ đã cho để gây lỗi

1. Test 1: Danh sách tham số hình thức rỗng, danh sách tham số thực sự có ít nhất 1 phần tử

- Code và kết quả:

```

tests > test_err1.kpl
1  PROGRAM TEST5;
2  VAR I : INTEGER;
3
4  PROCEDURE P;
5  BEGIN
6  END;
7
8  BEGIN
9    CALL P ( I );
10 END.

Problems  Output  Debug Console  Terminal  Ports  Spell Checker 112
● → Bai5_Type_Code_Gen ./semantics tests/test_err1.kpl
9-10:The number of arguments and the number of parameters are inconsistent.
○ → Bai5_Type_Code_Gen █

```

- Phân tích nguyên nhân:
 - Procedure **P** được khai báo **không có tham số**, nên paramList = NULL.
 - Khi compiler gặp lời gọi CALL P(I), hàm **compileArguments()** được gọi với paramList = NULL.

- Compiler parse dấu (và phát hiện có **đối số được truyền vào** trong khi procedure không có tham số.
- Do số lượng đối số và tham số **không tương ứng**, hàm **compileArguments()** báo lỗi **ERR_PARAMETERS_ARGUMENTS_INCONSISTENCY** với thông báo: *"The number of arguments and the number of parameters are inconsistent."*

2. Test 2: Danh sách tham số thực sự rỗng, danh sách tham số hình thức có ít nhất 1 phần tử

- Code và kết quả:

The screenshot shows an IDE window with a file named `test_err2.kpl`. The code is as follows:

```

1  PROGRAM TEST6;
2  VAR I : INTEGER;
3
4  PROCEDURE P ( V : INTEGER );
5  BEGIN
6  END;
7
8  BEGIN
9    CALL P;
10 END.

```

Below the code editor, the **Terminal** tab is active, displaying the following error message:

```

● → Bai5_Type_Code_Gen ./semantics tests/test_err2.kpl
9-8:The number of arguments and the number of parameters are inconsistent.
○ → Bai5_Type_Code_Gen

```

- Phân tích nguyên nhân:
 - Procedure **P** được khai báo có **1 tham số V**: **INTEGER**, nên `paramList` $\neq$ `NULL`.
 - Khi compiler gặp lời gọi **CALL P** (không có dấu ngoặc và không có đối số), hàm **compileArguments()** được gọi với `paramList` chứa 1 tham số.
 - Compiler kiểm tra token tiếp theo và phát hiện:
 - Không phải **SB_LPAR** (không có đối số).
 - Thuộc **FOLLOW set** như **SB_SEMICOLON** hoặc **KW_END**.
 - Do `paramList` $\neq$ `NULL` nhưng **không có đối số nào được truyền**, xảy ra lỗi **thiếu đối số**.
 - Hàm **compileArguments()** báo lỗi **ERR_PARAMETERS_ARGUMENTS_INCONSISTENCY**.

3. Test 3: Danh sách tham số hình thức chứa nhiều phần tử hơn danh sách tham số thực sự

- Code và kết quả:

```
tests > test_err3.kpl
1 PROGRAM TEST7;
2 VAR I : INTEGER;
3     J : INTEGER;
4
5 PROCEDURE P ( V1 : INTEGER ; V2 : INTEGER );
6 BEGIN
7 END;
8
9 BEGIN
10 CALL P ( I );
11 END.
```

Problems Output Debug Console Terminal Ports Spell Checker 112

● → Bai5_Type_Code_Gen ./semantics tests/test_err3.kpl
10-12: The number of arguments and the number of parameters are inconsistent.
○ → Bai5_Type_Code_Gen

- Phân tích nguyên nhân:
 - Procedure **P** được khai báo có **2 tham số**: V1 : INTEGER và V2 : INTEGER.
 - Khi compiler gặp lời gọi **CALL P(I)** chỉ với **1 đối số**, hàm **compileArguments()** được gọi để xử lý danh sách đối số.
 - Compiler parse đối số đầu tiên **I** bằng cách gọi **compileArgument()** với tham số **V1**, sau đó chuyển sang tham số tiếp theo **V2**.
 - Compiler kiểm tra token tiếp theo và phát hiện:
 - Không phải **SB_COMMA** (không còn đối số).
 - Là **SB_RPAR** (dấu đóng ngoặc).
 - Do vẫn còn tham số **V2** chưa có đối số tương ứng (currentParam ≠ NULL), xảy ra lỗi **thiếu đối số**.
 - Hàm **compileArguments()** báo lỗi **ERR_PARAMETERS_ARGUMENTS_INCONSISTENCY**.

4. Test 4: Danh sách tham số thực sự chứa nhiều phần tử hơn danh sách tham số hình thức

- Code và kết quả:

```
tests > test_err4.kpl

1  PROGRAM TEST8;
2  VAR I : INTEGER;
3      J : INTEGER;
4
5  PROCEDURE P ( V : INTEGER );
6  BEGIN
7  END ;
8
9  BEGIN
10     CALL P ( I , J );
11  END .
```

Problems Output Debug Console Terminal Ports Spell Checker 112

● → Bai5_Type_Code_Gen ./semantics tests/test_err4.kpl
10-14:The number of arguments and the number of parameters are inconsistent.
○ → Bai5_Type_Code_Gen

- Phân tích nguyên nhân:
 - Procedure **P** được khai báo có **1 tham số V** : INTEGER.
 - Khi compiler gặp lời gọi **CALL P(I, J)** với **2 đối số**, hàm **compileArguments()** được gọi để phân tích đối số.
 - Compiler parse đối số đầu tiên I, sau đó con trỏ **currentParam** được chuyển về **NULL** do đã hết tham số.
 - Khi gặp dấu phẩy **SB_COMMA**, compiler tiếp tục parse đối số tiếp theo trong vòng lặp while.
 - Do **currentParam == NULL** nhưng vẫn còn đối số cần xử lý, xảy ra lỗi **thừa đối số**.
 - Hàm **compileArguments()** báo lỗi **ERR_PARAMETERS_ARGUMENTS_INCONSISTENCY** vì số lượng đối số nhiều hơn số lượng tham số đã khai báo.

5. Test 5: Không tương ứng kiểu giữa tham số hình thức và tham số thực sự

- Code và kết quả:

```
tests > test_err5.kpl

1  PROGRAM TEST9;
2  VAR I : INTEGER;
3    C : CHAR;
4
5  PROCEDURE P ( V : INTEGER );
6  BEGIN
7  END;
8
9  BEGIN
10   CALL P ( C );
11 END.
```

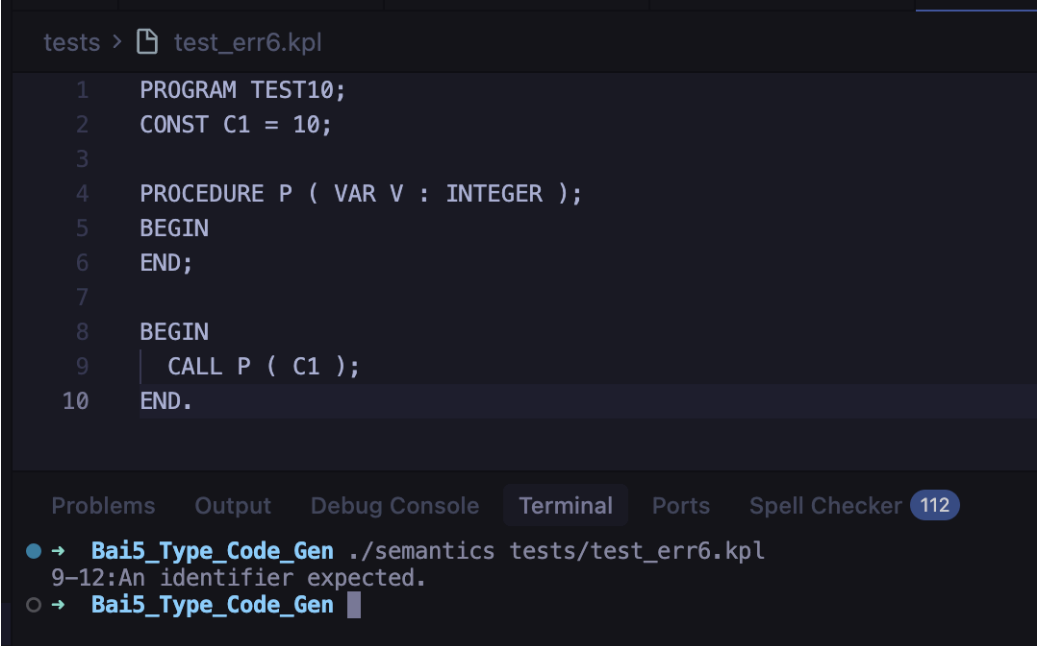
Problems Output Debug Console Terminal Ports Spell Checker 112

● → Bai5_Type_Code_Gen ./semantics tests/test_err5.kpl
10-12:Type inconsistency
○ → Bai5_Type_Code_Gen

- Phân tích nguyên nhân:
 - Procedure **P** được khai báo có **1 tham số V** : **INTEGER** (kiểu **TP_INT**).
 - Khi compiler gặp lời gọi **CALL P(C)**, trong đó đối số **C** được khai báo là **CHAR**, hàm **compileArgument()** được gọi để kiểm tra đối số.
 - Do tham số **V** là **PARAM_VALUE**, compiler gọi **compileExpression()** để parse đối số **C** và xác định kiểu là **TP_CHAR**.
 - Compiler so sánh kiểu của đối số và tham số bằng **compareType(TP_CHAR, TP_INT)**.
 - Do hai kiểu **không khớp**, **compareType()** trả về **0**, thỏa điều kiện lỗi.
 - Hàm **compileArgument()** báo lỗi **ERR_TYPE_INCONSISTENCY** với thông báo *"Type inconsistency"*.

6. Test 6: Truyền tham biến bằng 1 hằng biểu diễn bởi định danh

- Code và kết quả:



```
tests > test_err6.kpl
1 PROGRAM TEST10;
2 CONST C1 = 10;
3
4 PROCEDURE P ( VAR V : INTEGER );
5 BEGIN
6 END;
7
8 BEGIN
9   CALL P ( C1 );
10 END.
```

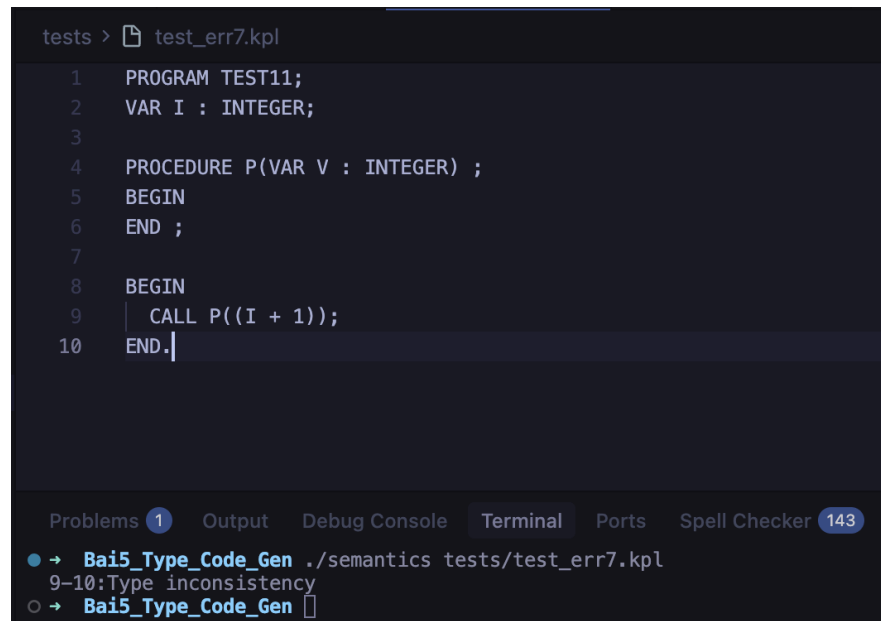
Problems Output Debug Console Terminal Ports Spell Checker 112

- → Bai5_Type_Code_Gen ./semantics tests/test_err6.kpl
9-12:An identifier expected.
- → Bai5_Type_Code_Gen

- Phân tích nguyên nhân:
 - Procedure **P** được khai báo với **tham biến** VAR V : INTEGER, yêu cầu đối số phải là **lvalue hợp lệ**.
 - Khi compiler gặp lời gọi **CALL P(C1)** với **C1** là **hằng số**, hàm **compileArgument()** xác định đây là **PARAM_REFERENCE**.
 - Compiler parse identifier **C1** và gọi **checkDeclaredLValueIdent()** để kiểm tra tính hợp lệ của lvalue.
 - Hàm **checkDeclaredLValueIdent()** tra cứu đối tượng **C1** và phát hiện obj->kind == OBJ_CONSTANT.
 - Do **constant** không thuộc các loại lvalue hợp lệ (**OBJ_VARIABLE**, **OBJ_PARAMETER**, **OBJ_FUNCTION**), kiểm tra rơi vào nhánh lỗi.
 - Hàm **checkDeclaredLValueIdent()** báo lỗi **ERR_INVALID_IDENT** với thông báo "*An identifier expected.*" vì hằng số không thể truyền cho tham biến.

7. Test 7: Truyền tham biến bằng 1 biểu thức

- Code và kết quả:



```
tests > test_err7.kpl
1  PROGRAM TEST11;
2  VAR I : INTEGER;
3
4  PROCEDURE P(VAR V : INTEGER) ;
5  BEGIN
6  END ;
7
8  BEGIN
9  CALL P((I + 1));
10 END.
```

Problems 1 Output Debug Console Terminal Ports Spell Checker 143

- → Bai5_Type_Code_Gen ./semantics tests/test_err7.kpl
9-10:Type inconsistency
- → Bai5_Type_Code_Gen

- Phân tích nguyên nhân:
 - Procedure **P** được khai báo với **tham biến** VAR V : INTEGER, yêu cầu đối số phải là **lvalue đơn lẻ**.
 - Khi compiler gặp lời gọi **CALL P((I + 1))**, đối số là **biểu thức trong ngoặc**, không phải lvalue.
 - Hàm **compileArgument()** xác định đây là **PARAM_REFERENCE** và kiểm tra token đầu tiên của đối số.
 - Compiler phát hiện token đầu tiên là **SB_LPAR** (dấu ngoặc mở), không phải **identifier**.
 - Do đối số không phải identifier đơn lẻ, điều kiện lỗi được kích hoạt ngay.
 - Hàm **compileArgument()** báo lỗi **ERR_TYPE_INCONSISTENCY** với thông báo “*Type inconsistency*” vì biểu thức không thể truyền cho tham biến.